

1 Introduction to Graph Traversal

A graph consists of vertices and edges. Graph traversal is the process of visiting the vertices of a graph in a systematic manner.

The two important graph traversal techniques are:

1. Breadth-First Search (BFS)
2. Depth-First Search (DFS)

Both techniques can be used to explore a graph, but they follow different visiting strategies.

- **BFS:** Visits vertices level by level.
- **DFS:** Explores as deeply as possible along a path before backtracking.

To avoid visiting the same vertex repeatedly, both algorithms maintain a **visited** array or set.

2 Example Graph

Consider the following undirected graph.

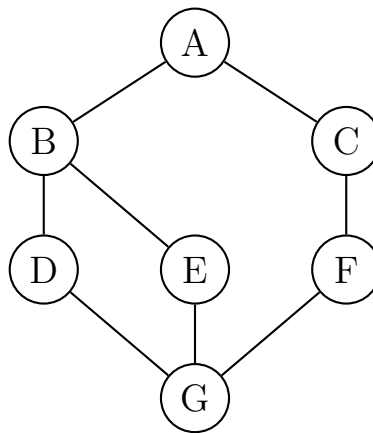


Figure 1: Example graph for BFS and DFS

The set of vertices is:

$$V = \{A, B, C, D, E, F, G\}$$

The graph is undirected. Therefore, every edge can be traversed in either direction.

For this example, start at vertex **A**. Whenever there is a choice, visit adjacent vertices in alphabetical order.

The alphabetical-order rule makes the traversal predictable. Different neighbor orders can produce different valid traversal sequences.

3 Adjacency List

An adjacency list records the neighboring vertices of each vertex.

For the given graph:

Vertex	Adjacent vertices
<i>A</i>	<i>B, C</i>
<i>B</i>	<i>A, D, E</i>
<i>C</i>	<i>A, F</i>
<i>D</i>	<i>B, G</i>
<i>E</i>	<i>B, G</i>
<i>F</i>	<i>C, G</i>
<i>G</i>	<i>D, E, F</i>

This representation helps us identify which vertices can be visited from the current vertex.

4 Breadth-First Search (BFS)

4.1 Definition

Breadth-First Search (BFS) is a graph traversal technique that visits vertices **level by level**.

It first visits the starting vertex, then all its unvisited neighbors, followed by the unvisited neighbors at the next level.

BFS uses a **queue**, which follows the First In, First Out (FIFO) principle.

4.2 How BFS Works

1. Select the starting vertex.
2. Mark it as visited and insert it into the queue.
3. Remove the vertex at the front of the queue.
4. Visit each of its unvisited adjacent vertices.
5. Mark each newly discovered vertex as visited and insert it into the queue.
6. Repeat until the queue becomes empty.

4.3 BFS Algorithm

BFS(Graph, start)

```
Create an empty queue Q
Mark all vertices as unvisited
```

```
Mark start as visited
Insert start into Q
```

```
while Q is not empty

    vertex = remove front of Q
    Print vertex

    for each neighbor of vertex

        if neighbor is unvisited

            Mark neighbor as visited
            Insert neighbor into Q
```

4.4 BFS Traversal: Step-by-Step

Start from vertex A.

Level 0

Initially, only A is visited.

A

Queue:

[A]

Level 1

A is removed from the queue. Its neighbors are B and C.

Both vertices are marked visited and inserted into the queue.

B, C

Queue after processing A:

[B, C]

Level 2

Remove B first. Its unvisited neighbors are D and E.

Then remove C. Its unvisited neighbor is F.

D, E, F

Queue after processing B and C:

[D, E, F]

Level 3

Remove D from the queue. Its unvisited neighbor is G. The remaining vertices E and F do not discover any new vertices, because G has already been marked visited.

G

The queue eventually becomes empty.

4.5 BFS Traversal Table

Level	Vertices	Explanation
0	A	Starting vertex
1	B, C	Neighbors of A
2	D, E, F	Newly discovered neighbors
3	G	Newly discovered neighbor

Therefore, the BFS traversal is:

A, B, C, D, E, F, G

Important: BFS visits vertices in increasing order of their distance in edges from the starting vertex. It does not necessarily visit vertices in alphabetical order unless that order is used when choosing neighbors.

5 Depth-First Search (DFS)

5.1 Definition

Depth-First Search (DFS) is a graph traversal technique that explores a path as deeply as possible before returning to an earlier vertex to explore another unvisited path.

This return process is called **backtracking**.

DFS can be implemented using:

- Recursion, which uses the program's call stack.
- An explicit stack data structure.

5.2 How DFS Works

1. Start at the selected vertex.
2. Mark the vertex as visited.
3. Visit an unvisited adjacent vertex.
4. Continue moving deeper while an unvisited neighbor exists.
5. If no unvisited neighbor exists, backtrack.
6. Continue until all reachable vertices have been visited.

5.3 DFS Algorithm Using Recursion

DFS(Graph, vertex)

Mark vertex as visited

Print vertex

for each neighbor of vertex

if neighbor is unvisited

DFS(Graph, neighbor)

Before calling DFS, mark all vertices as unvisited.

Then call:

DFS(Graph, A)

5.4 DFS Traversal: Step-by-Step

Start at A and visit adjacent vertices in alphabetical order.

Step 1: Visit A

Start at A.

A

A has two neighbors: B and C. Choose B first.

Step 2: Visit B

Move from A to B.

$A \rightarrow B$

B has unvisited neighbors D and E. Choose D first.

Step 3: Visit D

Move from B to D.

$A \rightarrow B \rightarrow D$

D has an unvisited neighbor G. Move to G.

Step 4: Visit G

Move from D to G.

$A \rightarrow B \rightarrow D \rightarrow G$

G has unvisited neighbors E and F. Choose E first.

Step 5: Visit E

Move from G to E.

$$A \rightarrow B \rightarrow D \rightarrow G \rightarrow E$$

E has no unvisited neighbors. Backtrack to G.

Step 6: Backtrack and Visit F

From G, the next unvisited neighbor is F.

$$A \rightarrow B \rightarrow D \rightarrow G \rightarrow F$$

F has no unvisited neighbors. Backtrack through G, D, and B as needed.

Step 7: Return to A and Visit C

B has no remaining unvisited neighbors. Backtrack to A.

The remaining unvisited neighbor of A is C.

Visit C:

$$A \rightarrow C$$

C has no unvisited neighbors because F is already visited.

The traversal is complete.

5.5 DFS Traversal Table

Step	Vertex visited	Action
1	A	Start
2	B	Move deeper
3	D	Move deeper
4	G	Move deeper
5	E	Backtrack after E
6	F	Backtrack after F
7	C	Return to A and explore C

Therefore, the DFS traversal is:

$$\boxed{A, B, D, G, E, F, C}$$

The traversal follows a deep path before returning to explore another branch.

6 BFS vs DFS

Feature	BFS	DFS
Full form	Breadth-First Search	Depth-First Search
Strategy	Level by level	Deep along a path first
Main data structure	Queue	Stack or recursion
Visiting approach	Explores nearby vertices first	Explores deeper vertices first
Backtracking	Not the main strategy	Essential to the traversal process
Shortest path	Finds shortest paths by number of edges in an unweighted graph	Does not generally guarantee shortest paths
Applications	Shortest paths in unweighted graphs, level discovery	Cycle detection, connectivity, topological sorting
Time complexity	$O(V+E)$ with adjacency lists	$O(V+E)$ with adjacency lists

7 Time Complexity

Let:

V = number of vertices

E = number of edges

When an adjacency list is used, each vertex is visited at most once, and each edge is examined a constant number of times.

Therefore, for both BFS and DFS:

$$O(V + E)$$

If an adjacency matrix is used, finding all neighbors of a vertex requires scanning a complete row of V entries.

Therefore, for both BFS and DFS with an adjacency matrix:

$$O(V^2)$$

8 Applications of BFS

BFS is useful when we need to explore nearby vertices first.

Examples include:

- Finding the minimum number of edges between two vertices in an unweighted graph.
- Finding all vertices reachable within a certain number of edges.
- Level-order traversal of trees.

- Exploring nearby locations in a network.
- Finding connections in social networks.

9 Applications of DFS

DFS is useful when we need to explore paths deeply before returning.

Examples include:

- Detecting cycles in graphs.
- Finding connected components.
- Topological sorting of directed acyclic graphs.
- Exploring mazes and backtracking problems.
- Checking whether a path exists between two vertices.

10 Important Note: Disconnected Graphs

If a graph is disconnected, starting BFS or DFS at one vertex will visit only the vertices reachable from that starting vertex.

To traverse every vertex in a disconnected graph, repeat the traversal from each vertex that is still unvisited.

```
for each vertex v in Graph
```

```
    if v is unvisited
```

```
        BFS(Graph, v)
```

```
        // Or use DFS(Graph, v)
```

In an implementation, BFS and DFS should share or update the same visited array when traversing a component.

11 Quick Revision

Question	Answer
What is BFS?	Visits vertices level by level
What is DFS?	Explores deeply before backtracking
Which uses a queue?	BFS
Which uses a stack or recursion?	DFS
BFS order for the example	A, B, C, D, E, F, G
DFS order for the example	A, B, D, G, E, F, C
Adjacency-list time complexity	$O(V + E)$
Adjacency-matrix time complexity	$O(V^2)$